

Lec 58 Practical Exercise - 1

Lec 58 Practical Exercise - 1

Lecture	58 / 63 + intro
Duration	48:27
Captions	English (auto-generated)
Words	7,477
Video	doc237ORyxc
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all, welcome to the hands-on series on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be learning the implementation of sentiment classification using two neural networks that is recurrent neural networks in short called as RNN and long short-term memory in short called as LSTM. So sentiment classification is a natural language processing task which deals with the text data. So in order to have the sentiment classification we take a database called as IMDb database which is the internet movies database comprising of details about movies. They were they are reviews uh actor information as well as some web series cal serals etc. So this is a NLP task or

[00:50] a natural language processing task which uh requires uh some kind of sequential data processing. So in order to have the sequential data processing, we need explicitly the RNN or the LSTM networks which deals with memory units. So they have the capability of remembering the information or the previous information. So which other neural networks do not possess. So in our implementation we make use of RNN and LSTM make their comparison see how the accuracy varies from RNN to LSTM see their hidden states and see also see the activation uh how the activation

takes place. So let us go through the implementation part the usual procedure importing the necessary libraries. So we have several uh libraries or their functions to be

[01:41] imported. So we start with numpy in short we call it as np numerical python required for processing the arrays or numbers. Importing matplotlib lib.pipplot as plt which is required for data visualization. Importing the tensorflow as tf because we need to work on uh deep learning models. So which is present in this tensorflow or the keras package. From tensorflow keras data sets we import the internet movies database. So in short but uh it is called as IMDb from TensorFlow KAS pre-processing sequence. We import a function called as pad sequence. So pad sequence is required in order to pad the sentences with zeros. Whenever the sentences are of ine unequal length, we have to make it as equal length. For that we uh pad

[02:29] with zeros. We can make use of either prepadding or postpadding. So in our implementation we make use of postpadding which we'll see very shortly and from tensorflow keras layers we import the different layers the input layers which makes use of the inputs or the input dimensions. Then we have the embedding layer where uh where the integers or the text is converted into integers. From the integers we have the vector representation for the words. So that is why this embedding layer is required whenever we deal with the text data. Then we have the simple RNN architecture. We have the LSTM layer which comprises of uh RNN network as well as the LSTM network. Then we have the dense layer which comprises the

[03:14] output layer. We have the drop dropouts. In order to drop the functioning of certain neurons, we need these dropouts as well as global average pooling one-dimensional. So as mentioned earlier, we convert the text into integers. those integers are converted into vectors. Now for every word there could be a vector. Now when we take certain sentences there could be combination of different words. So when we have different words there would be different vectors in order to have a single representation for a sentence. We can average those vectors. That's why we need this global average pooling function. And then uh from TensorFlow Keras models we also import the model in order to have the sequential models or

[04:01] layer stacking one after the other. And uh in order to have the performance metrics from scikitlearn we can have different metrics. So we need to import the accuracy score in order to have the accuracy validation accuracy test accuracy or training accuracy. Then to have a difference between the true label and the false label or the true label and the predicted label, we have this confusion matrix. Then the same matrix can be displayed. That's why we need a function called as confusion matrix display. And we have the classification report which comprises different metrics like accuracy, precision, recall and F1 score. So these are the different performance metrics we can have both for RNN as well as LSTM networks. And in

[04:46] order to adjust certain layers in the LSTM units, we need to uh print this TensorFlow version. So based on the version, we can have certain adjustment in the LSTM network. That's why we are printing this uh version. So once we print the version, we have this 2.20 version. So this is all about importing the necessary libraries. Soon after importing the libraries, then we come to loading the data set. So, internet movies database data set. It is a movies data set comprising of different movie reviews. So, we have such 50,000 reviews which are uh comprising of 25,000 training samples and 25,000 test samples. So, 20 5,000 reviews are going for training, 25,000 uh reviews are coming for testing.

[05:37] Now out of those reviews we set certain values or certain uh parameters like vocabulary size and maximum length. So vocabulary size refers to the number of unique words present in the data set. Now if we take 50,000 reviews in the movies data set there can be one lakh words or maybe 50,000 words or 80,000 words. But uh computational cost that comes with those many words will be too high. That's why we limit our vocabulary size to 10,000 so that we have 10,000 unique words. Those unique words will have unique code word nothing but unique uh integer representation. From the unique integer representation we have the unique vectors.

[06:22] Uh also maximum length is set as 200. We know that uh any reviews given or any sentence may vary in length. It is not a uniform length coding. uh instead it becomes a variable length coding but always we should remember that the neural networks need the input vector which which should be of same length. So we set the length as 200. So after setting these two variables we have this training set and the test set. So there are four variables X train Y train X test and Y test. So we load this IMDb data set into the four variables.

[07:00] number of words is same as the vocabulary size. So we have total number of words as 10,000 in our data set even though it has 80,000 different words. So now printing the training samples and printing the test samples as well. Uh it is based on the length of what is present in the X train variable and what is present in the X test variable. So when we print this we get the output as training samples as 25,000 and test samples as 25,000 once again. So totaling up to 50,000 samples or reviews.

[07:33] So now comes to uh coming to padding which is a pre-processing step. So what is done in padding? We pad the sequences that is we take the training set. We set the maximum length. So in our case maximum length was 200. So if we set the maximum length as 200 and padding is post padding. Beyond that if you want to truncate the sentence length we can also truncate and what we do is we make use of the post truncation. Same thing to do with the test set. We pad the test set and maximum length will be equal to 200. Uh once again we make use of post padding and post truncating process. So now when we print the X train and X test what happens is we have 25,000 reviews or samples in the training set and the

[08:28] length or uh what we say is number of columns present or number of features present are 200. So we have 200 unique words in the training set. Similarly in the test set we have 25,000 different reviews and 200 different features we can call it as variables as well. So let us exactly see with the help of an example what happens in pre-processing how we have a sentence how that tokenization takes place what is padding and what is uh uh vector formation. So let us take an example.

[09:00] So example of a positive sentence and a negative sentence. So as mentioned earlier we are performing the sentiment classification. The sentiment could be a positive statement or the sentiment could be negative in nature. Usually what happens is positive uh statement or positive uh sentence can be labeled as one, negative can be labeled as zero. So there could be two types of statements. One is positive, other one is negative.

[09:26] So we label it as 1 and zero. Now what if we take the first example for a positive statement? The statement could be something like this. The movie was interesting. Actors acted well. So this is one particular review. So the word says that this is a positive statement. So label could be one. Let us take the second example. movie was boring. Maybe climax was dull due to which I did not like the movie. This could be an example of a negative statement labeled as zero. Now when we look into the two statements here, the number of words present in both the reviews are different. So if I consider the first example, the movie was interesting, actors acted well. So there are seven different unique words. So the

[10:54] length will be equal to seven. But whereas if I take the second statement, then movie was boring. to 3 4 5 6 7 8 9 10 11 12 13 14 15 There are 15 different words. Now they are of unequal length. When we have unequal length, the neural networks will not accept such inputs. So what we need to do is this seven unique words and 15 unique words should be made of equal length. So what if I have to make equal length? If I take maximum length as 20 then after this I need to pad after these seven words I need to pad zeros.

[11:35] So how many zeros until the length becomes 20. So finally I should have 20 different unique words. Now there are 15 already 15 words present. How many zeros I need to append? Five more zeros I need to append so that the length becomes 20. So this is called as padding. So padding of zeros. So what we have performed here is post padding because I have included the sentence prior after the end of the sentence uh we have padded the zero. So the other method is first take the number of zeros how much ever a zero is required and then take your sentence. Same thing to do with the second example. Take five zeros and then uh take the input. So both the cases the input length will be equal to 20. So either we perform

[12:27] prepadding or postpadding. So the importance of post padding is that we first give importance to the input what is present. So only when uh we need to match

the number of words we uh we take the input as zero. So what happens is activation is more uh likely to be present at the beginning of the statement and gradually when the values are zero or the dense vector values are zero there is no neurons which are activated. So this is the possibility in case of post padding of zeros. So now every sentence we have converted into unique words. So that is called as tokenization.

[13:08] So initially we had sentences which we converted into different words called as tokens or this process is called as tokenization. After tokenization we give certain uh integer representation to it. So some uh code word representation or we can call it as encoding. So every word maybe the word if I take the word as the the can be represented with the unique uh integer maybe three. Movie can be given another uh integer maybe 10.

[13:46] Likewise different integer representation we will give after we have these code words for every code word maybe the code word three maybe the code word 10 that is represented as different features once again. So h how we get this feature representation is converting that into a dense vector. So our code word three is now converted into a vector maybe call we call it as a dense vector as well. So we should be having certain neurons in order to represent these values that is set either in the RNN layer or in the LSTN layer. So that we will see in the implementation part.

[14:29] So this is all about pre-processing. Now coming to building the RNN. So what is the set of inputs? So we have the input layer and the shape is dependent on the maximum length. In our case the maximum length will be equal to 200. Soon after the input layer we have this embedding layer. So this is where we have the input which is sentence converted into tokens converted into integers and then we have the 128 different dense vector representation.

[14:59] So what happens is input dimension will be same as vocabulary size. So vocabulary size is 10,000 different unique words what we have taken and the output dimension for every unique word that is present we want 128 features to be extracted. So we have 128 different values. So those 128 values are now down converted or represented only using 64 different neurons. So we try to compress it here so that the computation becomes little less here. So 128 different tense values are represented with the help of 64 different neurons. Nothing but 64 different activations. Now based on whether the sentence is showing some positive uh kind of nature or negative kind of nature certain neurons will be activated. We have different neurons for

[15:47] different purpose. Maybe your neuron one maybe neuron number one or hidden neuron one can understand only the positive words. Hidden neuron 2 may understand only the negative statement. Hidden neuron 3 may understand the relationship between the two words. Maybe hidden neuron 4 will focus only on the adjectives present. Likewise, we have different neurons for understanding the different words. Maybe their nature, the their characteristic, their dependency and so on.

So, we have the simple RNN comprising of 64 neurons and let us give the name to this uh RNN architecture or RNN layer as RNN layer which is acting upon the input X. So finally we have the dense layer and we need only one neuron because this is a binary classification

[16:40] problem. We have only two kinds of reviews reviews either the positive review or the negative review. That's why we need only one neuron and uh as we have seen in our previous hands-on whenever we have binary classification we need to make use of the sigmoid activation function because it gives values between the range 0 to 1. Whenever the probability of occurrence of any uh value or word is greater than 0.5 we take it as one class. If it is less than 0.5 it is the other class.

[17:09] So in our case what we try to do is whenever the values are greater than 0.5 we call it as a positive remark or review and whenever the values are less than 0.5 we call it as a negative review. So this is about the input layer, the m uh embedding layer, the RNN architecture and the output. So finally we build this RNN model based on the inputs as well as the output. Now configuring the model that is uh compiling we make use of the optimizer called as Adam optimizer. We have the binary cross entropy as a loss function and let us uh measure the accuracy accuracy training accuracy or validation accuracy or sometimes even the test accuracy that is what we try to measure.

[17:55] So now coming to training the RNN model. So we fit this RNN model on the training data set comprising of two variables the X train and the Y train. uh let me try to maintain 10 epochs and let the batch size uh number of uh uh reviews that have to take at a time is 64 different reviews and let the validation split be 20% 20% of the data going for validation. So when uh it is run for 10 different epochs we are able to see the training accuracy as well as the validation accuracy. Similarly the training loss and the validation loss. So when uh coming to the end of the 10th epoch the validation accuracy happens to be only 50%. And the validation loss is pretty high that is 1.26.

[18:43] So in case of RNN what happens is the accuracy will be slightly less than LSTM because RNN has only one hidden memory or hidden unit due to which it will not be able to capture the long-term dependencies between the words. So whenever we talk about text data, it is a time series data or it is a sequential data wherein the current word is dependent on the previous word or sometimes we can say the current word is also dependent on the future word. So that is why we have different types of LSTM maybe a bidirectional LSTM. So whenever there is dependency, the RNN fails a little bit because of the vanishing gradient problem because it is not able to capture the long-term dependency. That's why it is very

[19:30] evident that the validation accuracy is half that is it is only giving us 50%. So when we try to evaluate the RNN, let us see what kind of prediction it is doing

for us. So RNN model prediction on X test data. uh whenever the prediction value or whenever the vector value is greater than 0.5 then it is class one nothing but a positive statement whenever it is less than 0.5 it is a negative statement so RNN accuracy based on Y test and whatever prediction that is uh Y pred or RNN prediction uh variable so when we have the difference between them and we try to print the accuracy the accuracy is only 50%. So let us see how it is only 50% for us. So we make use of the confusion matrix in order to have the difference between the

[20:24] true label and the predicted label. So confusion matrix acting upon the y test and the RNN prediction uh variable. We are trying to display the confusion matrix uh nothing but displaying the plot as well as showing the plot. So we have the correct classifications as 8,846 plus 3,672. So what is happening here? Uh the true label is negative and whatever prediction done is also negative. So this is negative statement and negative uh which is predicted negatively by the model. Same way one refers to this 3,672 refers to true labels being positive and model prediction is also positive. Now coming to 3,654 true label actually uh in the data set it was a negative statement that's why we have the label as zero but model has

[21:24] predicted it as one. So this is a set of wrong classifications. Similarly, we come to this uh value. We have the true label as positive, but model has predicted it as a negative statement. So, predicted label is zero. That's why we have 8,828 such wrong classification. So, when we sum up this, these refer to the true classification. When we sum up these, the value refers to the wrong classifications uh done by the model.

[21:53] So, we see that only 50% of the data or classifications are right. So now we try to move on to the second model uh which works on the sequential data which is the longsh short-term memory. So we will try to build the LSTM model. Once again uh it the layering will be same. We have the input layer followed by the embedding layer and then we have the LSTM layer. Finally the output layer. But just to uh you know play around with the neurons or the hidden neurons because the number of hidden neurons are more here or the number of hidden state is more when compared to RNN. RNN has only one hidden state that's why we try to play around with the neurons and that's why we have the dropout function here. We try to

[22:41] inactivate certain neurons. So let us see this uh building of LSTM. We have the input layer which takes the shape of maximum length. Maximum length is 200. So 200 unique words present in any review. And then we have the embedding layer which makes use of the input dimension which is same as vocabulary size. So total number of unique words in our data set are only 10,000. And let us have 128 different features extracted from those 10,000 words or 50,000 reviews. And let the name of this layer be embedding layer.

[23:16] Soon after this embedding layer wherein we try to convert the sentence into

words and one words into feature representation we have this LSTM which takes 128 different features as 64 different hidden values or 64 different hidden neurons and here we make use of written sequence equal to true that means we try to capture the dependency between different words so the current word should depend on the previous words that's Okay, we keep this written sequence equal to true. The sequence shows should show some dependency so that we predict the next word. That's why we have this written sentence return sequence is equal to true. Let me give a name to this LSTM layer uh as LSTM layer. Soon after which we have the

[24:07] global average pooling. So as mentioned earlier for 128 different for for 200 different words what we have we have different dense vector representation. So 200 words are converted into 128 different variables. Now those 128 variables are converted into 64 different neurons or representation. Since we have different representation for a word. Now we try to have one dense vector so that the network is not confused. During back propagation we should have updation of only one weight.

[24:44] That's why we do this global average pooling. We convert the 64 different representation as one representation. That's why we average the dense values or dense vector values. Soon after that we will try to inactivate 50% of our neurons. And finally at the output layer we have the dense layer one neuron which is going to give us either it is a positive statement or a negative statement. And once again the activation function is sigmoid.

[25:14] So our LSTM model is built based on the input dimensions that we have taken and the output dimension which comprises only of one neuron. Now let us configure this LSTM model uh based on the optimizer which is the adam optimizer. Since it is a binary classification problem, we have the binary cross entropy and let us measure the accuracy. So while training the LSTM, we fit it into X train and Y train. Let the number of epochs be 10 and the batch size is equal to 64 and validation split as 20%.

[25:48] So the same uh arguments or the same values we have trained to maintain even in the RNN so that we have a comparative study between RNN and LSTM. So when we have run this at the end of the 10th epoch we see that the validation accuracy happens to be around 85%. It is 8482 and validation loss is 77 unlike the previous case it was above 1. So LSTM has performed better when compared to RNN. So let us see the reason behind the LSTM performing better when compared to RNN.

[26:26] So in case of RNN what happens is we have the previous hidden state soon after which we have the current word and then we have the simple RNN layer and from the RNN layer we have the new hidden state. So this will give us only one hidden state. That is why it is not able to capture the long-term dependencies. Whereas in case of LSTM we have the previous hidden state from which we have the previous cell state also. We have the current word which makes use of both the hidden state as well as the previous cell state.

[27:46] We have three gates acting upon this current word that is the forget gate, the input gate and the output gate. So the forget gates takes into account how much information has to be forgotten and how much to be passed on to the next hidden layer. So this is one important gate which controls the information flow within the one uh LSTM unit or we can call it as one hidden uh unit. And now from because of these uh three gates we have the new hidden state as well as the new cell state.

[28:36] So hidden state corresponds to the short-term dependency or short-term output we can call. And this refers to the long-term output. When we have the combination of both the short-term output and the long-term output, we have the name LSTM. That is long short-term memory. So due to this nature of LSTM, we have the LSTM working comparatively better when compared to this RS RNN. So now coming to the evaluation part of LSTM. LSTM making prediction on the X test data. Once again the constraint remains the same. Whenever the predicted value is greater than 0.5, consider it as one of the class which is a positive class. When it is less, consider it as negative class. So now let us display this accuracy score which is based on uh

[29:27] the actual output that is Y test and one which is predicted by the LSTM. So what is the LSTM accuracy? It is h happening to be 82 uh or 82%. Now let us display the confusion matrix. Once again uh the code or the functions used is same that is used in case of RNN confusion matrix acting on the y test as well as uh the output label which is predicted by the LSTM. We have the confusion matrix display uh we have this plot display. So when we see this on the rows we have the true label and on the columns we have the predicted label. So when we see this the number of classification the true classifications are more it is above thousand or above 10,000 when compared to the wrong classification which is just 4,000 uh in

[30:17] number. So let us now have the validation accuracy plot and validation loss plot both for RNN as well as LSTM. So we make use of the figure function and the argument figure size uh width being 8 in and height being 5 in. So plot which is dependent on history of RNN. So what what is this history of RNN? It takes into account the validation accuracy. Uh let the marker at each of the epochs be O marker. So at every epoch we have different accuracy values as well as loss values. Let us represent it using a marker that is the O marker and the label given to this uh plot as simple RNN. Same thing to do with LSTM. Let us have the plot of validation accuracy for LSTM. Let us make use of the square marker. So the label is LSTM. On the

[31:10] X-axis, let the label be epoch. On the y-axis, let the label be validation accuracy. Let the title be also called as validation accuracy. Uh let the grid lines be present. So grid lines are these uh square boxes or rectangular boxes that we are able to see in gray shade so that we can have uh a correct uh demarcation between the values. That's why we make use of the grid grid being true and legend used to give the comparison between two uh different architectures. The RNN represented as blue color and LSTM being represented as orange color. So when we show this

graph on the x-axis we have the epoch we have run this for 10 epoch so 0 to 10 we have and on the y-axis we have the validation accuracy uh it is clearly evident that

[31:59] LSTM has a highest accuracy somewhere around 83% 82 or 85 at times for when we have run for 10 different epochs whereas uh RNN uh has values somewhere around 50%. Now when we have the validation laws we takes take into the history of validation laws both for RNN as well as LSTM. Let the markers be O marker and the square marker only uh changes in terms of the labeling uh Y label being validation loss and title being validation loss. So when we uh show this graph here we see that for LSDM which is represented in orange color the loss is less somewhere around 4.3 and 2 but whereas in case of RNN uh the value increases and it can be seen that it is higher when compared to the LSTM. So loss is higher for RNN when compared to

[32:58] LSTM model. Now coming to the important part that is the hidden state model. Why this hidden state model is required is we have seen in case of both RNN as well as LSTM. Initially we have the input layer followed by the embedding layer. Then we have RNN or LSTM layer and finally the output layer or the dense layer. So exactly what is happening in this LSTM layer or what is this hidden state or what is this RNN layer that is indicated with the help of hidden state. So hidden state makes use of the LSTM model input.

[33:35] The input and the output will be same but we stop at the LSTM layer. So what we uh do is in case of RNA we have the input layer followed by the embedding layer. Then we have the RNN layer and then the output layer. In case of LSTM, we have the input layer followed by the embedding layer, then the LSTM layer and then the output layer. So when we take input output layer or input and output in this case, we know input was the review that was given and output is in terms of number. Maybe if it is greater than 0.5, we call it as class one. If it is less than or equal to 0.5 we call it as class 0 positive and negative. Same thing applicable even for LSTM. So what if we stop at the LSTM and try to figure

[34:33] out what exactly is happening at this layer or what if we stop at the RNN layer and we try to figure out where exactly the activation function where exactly the activation or the neuron activation is taking place. So when we stop at LSTM or RN L layer this is the hidden state output. So that is what we are trying to do. So in both the cases the input will remain the same. So whatever input given to LSTM as well as RNN same thing holds good even for the hidden state. Uh so inputs will be same as LSTM model input. Why take why do we take LSTM is because LSTM has performed better. Same thing we can do even with RNN outputs will be getting the information from the output that is the previous layer. So we call it as LSTM model dot

[35:23] get layer from which layer we have to get the information from the LSTM layer. So we have to get the output from the LSTM layer. So now what is the hidden state? So let us see the hidden state for one of the sample or one of the review present

in the data set. So let me take the zeroth index sample X test for the zeroth index. So E is to1 means it is the zeroth sample. So I call it as sample. Here I want to predict the hidden model. From the hidden model, I want to predict the hidden state. So what is the hidden state? When we print the shape of the hidden state, we know that we have taken only one sample. we had 200 unique words in that sample and 64 neurons working on that particular sample. So this is the shape. Now when

[36:14] we try try to have a heat map of that hidden state uh in order to have that let us take figure function. Uh let us show the image. Okay hidden state for the zeroth index. I take to I try to take the transpose of it. Transpose can be taken or need not be taken. When we take uh try to try to take the transpose what happens is on the on the rows we have the hidden units on the columns we have the word position unlike what we have done is we had 1 comma 200 comma 64 that is for one sample we had 200 words and 64 neurons. So 64 hidden functions or hidden uh states we had what in order to have the heat map instead of having 200 word position and 64 when we take the transpose this becomes 64 different hidden states and 200 word

[37:16] position that's the difference. So for 200 different word positions we have the hidden units. So let us see the value here. the value varies from minus1 to 1 because we know that LSTM makes use of the tanh activation function tanh the values vary from minus1 to 1. So here we have different combination of values that is uh we have taken the color map as veridis where the values could be something like purple the colors could be something like purple green blue and yellow shades. So different shades have different meaning. So we see that purple color means it is a negative activation.

[37:56] Somewhere around yellowish means it is very positive activation. When we have values around green and blue it is no activation or small activations which are happening. So activation means we can say that how much the neuron is trying to learn or what information the neuron is trying to pass whether it is some positive statement being occurring or a negative statement being occurring or what is the pattern that has been recognized. All that depends on what is learned by a neuron or the activation.

[38:26] So based on the colors we will be able to see how much of negative activation is taking place, how much positive activation and how much no activation. So if we take any word, so if we take uh the word positions from 0 to somewhere around 20 the activations are lying between minus.2 2 to2 because most of the values are like uh most of the shades are like green and blue. So what is the indication the model is trying to learn or capture the information. Now gradually after 20 if you see up to 55 or maybe up to 60 there is lot of variation in the color because the review as the review happens somewhere in the middle middle of the statement there is lot of learning which is happening. So the activations are

[39:15] sometimes becoming positive, sometimes becoming negative. That's why we have the difference in the color. Now after maybe 60 or 70 the color is trying to be same. If I take the first if I take the zeroth row here, it is happening to be very purple and it is constant throughout the end of the 200 position. Similarly, if I take the next one, the green is trying to be constant up to the 200 position. That is because any review sometimes the sentence can be 100 word long or 60word after 100 words we have just appended zeros. So there is nothing learning by the neuron on that zero values. That's why it is going to be constant. We have completely yellows or completely green here. So after maybe 60

[40:04] 70 or maybe uh up to 75 there is some learning happening. After that it is going to be constant. So this is what is happening in the hidden unions. So we are trying to show based on the review based on the length taken uh in the review statement what kind of learning is happening how the activations are varying with respect to every neuron 64 different neurons present in the uh uh hidden uh present in the LSTM or the RNN model.

[40:38] So we can also make use of three hidden neurons and display what exactly they have learned. So hidden unit evolution uh trying to show it in the form of a figure. So label is unit one, label is unit two and unit three. That means we are trying to show how what activation or what information is learned by hidden unit one, hidden unit two and hidden unit three. So here uh 0 is to zero means we take only one sample and 200 words is to means it refer refers to 200 words like that I want only for the first sample or the zero sample. Similarly I want for the first sample and for the second sample.

[41:23] So when I try to do do this X label being word position Y label being activation title being evolution of hidden units. Once again on the x-axis we have 200 word positions and the activations on the y-axis. Maybe up to 70 70 or 75 there is some learning happening and maybe the review length is just 70 or 75. After 75 we see that it is becoming very constant. Either the blue uh uh shade is becoming very constant and there is no learning. There is no deviation. It is remaining stagnant. Same thing to do with unit two or unit 3. There is difference in learning that can be seen up to from 0 to 75. So that is very much evident here. Sometimes the value become negative, sometimes it it is becoming

[42:13] positive. So what is this negative and positive value means? Suppose we take a hidden uh neuron one which focuses only on the positive word. So whenever it is getting positive word the value will be above 0.5 whenever it is getting some negative values negative statement the value decreases for that neuron. So that variation is making it fluctuate either from positive side to negative side. So this is how the hidden state uh evolution takes place. Final uh uh coming to the final stage of this implementation predicting the review. So let us take two samples one for positive statement other one for negative statement and let us also see what the statement

was whether the prediction is rightly done by the LSTM model or no. So

[43:03] let us take zero test sample uh and one sample we are taking minus one refers to how 200 words have to be present there or how it has to be arranged in the form of 200 different words. So once again we are making use of the LSTM model on the review. We are printing the prediction based on whether the prediction is greater than 0.5 or less than 0.5. So whatever dense vectors present those we will be converted into some probability value at the dense layer with the help of the sigmoid activation function. So when it is greater than 0.5 it's a positive statement. Whenever it is less than or equal to 0.5 it is a negative review. So what has happened at zero sample? It is saying that the prediction

[43:46] uh value is 0.01 that happens to be a negative review. Now same thing we will take the 20th sample. 20th sample the value is .99. Okay. And that's why the it is a positive review. Now let us see what is the zeroth and the 20th sample. So when we try to display the review the word index from the IMDb get the word index. That means now it is a reverse engineering. Earlier we have taken a sentence. We have got the dense value or the probability value based on which we have done the prediction. Now based on the dense vector value we have to go back to the integer. We have to go back to the token and then predict the sentence or get back the sentence. So this is a reverse way what we are doing.

[44:36] From the value we have to get the keys. So that means we are defining a dictionary. Here we have the prediction probability that is a value we have to get the key for it. That's why uh we are trying to get three three uh keys for it. So uh what is this three different values that is 0 1 2 3 what are the values? So one is we should consider how much padding we have done. How what is the beginning of the statement? How many unknown words are there? Because I already mentioned that this IMDb data set has 80,000 different vocabulary size or uh the vocabulary size 80,000 that means 80,000 words are present unique words but we have considered only 10,000 words. So some of the words may be unknown to LSTM or RNN. So what we need

[45:25] to do is in those places wherein the uh RNN or LSTM is not able to predict that particular uh word in those places let us write unknown. So that's why the representation is u n k. Now what is word index three? There could be some kind of unused words. The words may not be used in the reviews at all. None of the test samples we have come across those words. So such words can be called as unused words. So let us join whatever has been padded at the beginning of the statement some unknown words which are present in the reviews or the unused words also. So we are joining and what we are trying to do is reversing reversing the process. So we from the uh prediction value we are trying to get the word index. So when we print such a

[46:16] review for the zeroth test sample let us see the sentence. It is the start of the sentence. Please give this one a miss. Something is unknown to the model and the rest of the cast render terrible performance. The show is flat flat flat. So with this itself is it it is understood for human beings it is understood that the statement is a negative statement because it's a terrible performance. The show is flat that's why the review earlier was negative. So we have seen this for the zero test sample the negative review has appeared.

[46:53] Now let us see for the 20th sample the same thing the same code has been run for the 20th sample. So beginning of the statement or starting of the statement this film was one that I have waited to see for some time. I was glad to find it has been uh it has been everything anticipated. The writing of the film has been so finely crafted. So it is known that it's a positive statement. That's why for the 20th sample LSTM has predicted it as a positive review.

[47:27] So this is all about sentiment classification on IMDb data set. So what we have done is we have taken two neural networks which works well on the sequential data that is the RNN and the LSTM model. We have built both RNN and LSTM model. We have tried to make a comparative study between the accuracy and the loss. And we have uh seen that LSTM performs better in CA uh in case of accuracy or the loss because LSTM has input forget and output gates. With the help of forget gate it it knows how much information to be forgotten to be lost and how much dependency is present for a current word based on the previous words. So, so using the hidden state as well as the cell state, LSTM is able to perform better when compared to RNN. So,

[48:16] this is all about sentiment classification on IMDb data set. Thank you.